

Tema del sito:

Il sito AdsSite ha come obiettivo permettere ad un utente registrato di poter vedere gli annunci disponibili sulla piattaforma potendo usufruire della ricerca, a cui può accedere dopo aver fatto l'autenticazione/registrazione, e poter disporre di varie funzionalità che un utente non registrato non può usare come ad esempio poter inserire dei propri annunci, visualizzare l'elenco degli annunci creati e creare la propria lista dei preferiti con gli annunci trovati nel sito stesso e poterli eliminare dalla lista stessa se è opportuno.

Sezioni principali:

index.html:

- permette di accedere al sito principale MainpageUtente inserendo l'email e la password nelle apposite caselle.

Index_registrazione.html:

- la pagina in questione permette all'utente di registrarsi al sito inserendo la propria email, username, nome, cognome e password con dovuto controllo su di essa inserendola una seconda volta per la conferma.
- Registrandosi l'utente verrà indirizzato nella pagina principale MainpageUtente.

mainpageUtente.php:

- Questa pagina è solo accessibile a chi è registrato nel sito e dispone di diverse funzioni fatte per l'utente:
 - Ricerca, l'utente sulla base di una parola chiave, zona e categoria dell'annuncio può fare una ricerca che gli stamperà un elenco di annunci che corrisponderanno alla richiesta fatta.
 - Inserimento annuncio, l'utente ha la possibilità di poter inserire degli annunci propri mettendo informazioni riguardanti il nome, la descrizione del prodotto, la quantità, una immagine, prezzo, categoria e zona a cui è associato.
 - Elenco annunci personali, l'utente può visualizzare gli annunci personali inseriti.
 - Lista preferiti, lista apposita dove l'utente può salvarsi gli annunci che ha gradito e poterli eliminare quando non gli interessano più.
 - Log out, permette all'utente di effettuare il log out.

Admin_zone.html:

- Pagina usata dall'admin del sito in cui ha la possibilità di poter visualizzare gli utenti registrati nel sistema e poterli eliminare nel caso di necessità e poter visualizzare gli annunci totali nel sito e poterli eliminare.

Funzionalità:

Login/logout: La pagina HTML ha un modulo che include campi per inserire un indirizzo email, password e un pulsante per inviare il modulo.

L'attributo "action" del modulo specifica che i dati dovrebbero essere inviati alla pagina login.php quando viene cliccato il pulsante.

Nel momento in cui viene cliccato il pulsante avviene un richiamo a una funzione javascript chiamata controlloLogin() dove viene verificato se i campi sono stati compilati e se nel caso non lo fosse stamperebbe un messaggio di errore, in caso contrario avviene il passaggio alla pagina php che comunica con il server con dovuti controlli sulla autenticità dei dati.

Nella pagina php viene chiamato "core.php" che contiene le funzioni per la connessione a un database. Quindi, assegna alle variabili \$email e \$password i valori inviati nel modulo di login, ma prima utilizza la funzione "mysqli_real_escape_string" per "scappare" i caratteri speciali nei valori, il che protegge lo script dalle possibili injection di SQL.

Successivamente, esegue una query SQL per selezionare la password hash dal database per l'utente con l'email specificata.

Quindi, utilizza la funzione "password_verify" per verificare che la password inserita nel modulo corrisponda alla password hash nel database.

Se le password corrispondono, l'utente viene considerato autenticato e lo script inizializza alcune variabili di sessione per l'utente (come l'ora di login, lo stato, l'email e l'ID dell'utente) e reindirizza l'utente alla pagina "mainpageUtente.php" e se le password non corrispondono, lo script reindirizza l'utente nuovamente alla pagina di login.

Infine, chiude la connessione al database.

Registrazione: La pagina presenta un modulo per la registrazione degli utenti e ha campi per permettere all'utente di inserire il proprio nome, cognome, username, email, password e conferma password.

Nel momento in cui viene cliccato il pulsante avviene un richiamo a una funzione javascript chiamata controlloSumit() dove viene verificato se i campi sono stati compilati e se hanno la giusta sintassi e lunghezza; Se nel caso non lo fosse stamperebbe un messaggio di errore in base alla condizione non rispettata, in caso contrario avviene il passaggio alla pagina php che comunica con il server con dovuti controlli sulla autenticità dei dati.

Nella pagina php che comunica con il server, ovvero registrazione.php, viene gestito l'invio di un modulo per creare un nuovo account utente.

Quando il modulo viene inviato, lo script leggerà i valori del modulo di input, li sanitizzerà per proteggere contro gli attacchi di SQL injection utilizzando mysqli_real_escape_string(), cifrerà la password utilizzando password_hash() e inserirà i nuovi dati dell'account utente in una tabella del database chiamata "iscritto". Lo script avvia anche una nuova sessione e imposta alcune variabili di sessione, come 'logged', 'username', 'id_iscritto', 'nome' e 'cognome'.

Infine, lo script reindirizza l'utente a una pagina chiamata "mainpageUtente.php" in caso di invio riuscito.

Gestione del contenuto generato dall'utente: Tutto il contenuto generato dall'utente, a partire dai dati del login, registrazione e degli annunci generati dagli utenti stessi viene salvato in un apposito db chiamato Adssite.

Un esempio concreto di come viene gestito il contenuto dell'utente possiamo vederlo attraverso la funzione che permette di inserire e eliminare un annuncio nella lista dei preferiti dedicata per ogni utente.

Attraverso una immagine con il simbolo “+” ,accanto a ogni annuncio stampato attraverso il file “ricerca.php”, avviene il richiamo alla funzione javascript inserimentoprefe() con un passaggio come argomento all’interno di essa della variabile id_annuncio, che attraverso questa riga di codice presente in ricerca.php: `$a=mysqli_fetch_array($rs);` mi recupera una riga di risultato come array associativo così permettendo di poter passare in fase di ciclo di stampa degli annunci l’id_annuncio che identifica lo specifico annuncio selezionato dall’utente cliccando sul “+”.

Nella funzione javascript avviene una richiesta http tramite AJAX dove viene inviata tramite “open” e “send” dell’oggetto XMLHttpRequest e questa richiesta viene inviata al file php “inserimento_lista_prefe.php” con un parametro di query “id_annuncio” che ha come valore l’argomento della funzione.

Attraverso la pagina “inserimento_lista_prefe.php” viene eseguita una query SQL per inserire un nuovo record nella tabella “listaprefe” del db passandogli “id_iscritto” (id utente in sessione) e “id_annuncio” (id annuncio passato come argomento dalla funzione js) e seguentemente andando nella pagina index_lista_preferiti.php potrai vedere l’annuncio inserito tra i preferiti.

Poi nella lista dei preferiti ce un sistema analogo anche per eliminare quell’annuncio dalla lista.

Caratteristiche:

Usabilità: Per una buona usabilità si è optato per rendere i form dove vengono inseriti i dati per l’utente e dove avviene una interazione diretta con l’utente tramite pulsanti il più intuitiva e semplice possibile, per i form riguardanti la registrazione, il login e l’inserimento dati esiste un pulsante apposito visibile che permette di inviare i dati al server per poterli gestire e controllare in modo semplice.

Interazione/Animazione: Mentre per le interazioni e animazioni tra i vari esempi presenti nel sito è presente un pulsante adibito al controllo e all’invio dei dati nel server che è impostato con una animazione in diversi stili css.

Con questi stili css il cursore quando si trova sopra l’elemento pulsante l’elemento span dovrebbe avere un padding di 25 pixel a destra e un after pseudo-element (“») dovrebbe apparire a destra dell’elemento span con un carattere freccia (») e un effetto di transizione.

Quando il pulsante è attivo (cioè, viene cliccato), avrà un’ombra di casella e un leggero effetto di trasformazione verso il basso.

Tra gli esempi di iterazione abbiamo quello dei simboli immagine adibiti alla aggiunta e eliminazione dell’annuncio dalla lista dei preferiti che vengono rese impercettibili agli occhi dell’utente a livello di lavorazione dell’operazione e una volta che l’operazione interna viene eseguita avviene sempre la ricarica automatica della pagina per segnalare il successo di essa.

Sessioni: I dati dell’utente, soprattutto del suo id_iscritto tenuto nella tabella iscritto del db, verranno salvati in \$_SESSION in fase di sessione di rete nel momento in cui si iscrive al sito o fa il login in modo da poter memorizzare nelle funzionalità seguenti della pagina principale il contenuto dell’elenco annunci personali o di lista dei preferiti a nome dell’utente loggato sul sito.

Dopo l’accesso o registrazione, ogni singola pagina seguente dalla mainpageutente alle sue funzioni/pagine viene incluso un file php (session.php) che imposta la durata di una sessione a 30 minuti e avvia una nuova sessione; Quindi controlla se una variabile di sessione chiamata \$sessionid non è uguale a true (impostata precedentemente in fase di session_start() nel login o registrazione).

Se non è true, l’utente viene reindirizzato alla pagina index.php e inoltre il codice controlla anche se il tempo trascorso dall’accesso dell’utente è maggiore della durata della sessione, Se è così l’utente viene reindirizzato alla pagina index.php e lo script esce; Ciò viene fatto per assicurare che l’utente venga disconnesso dopo che è passato un certo periodo di tempo dal momento dell’accesso.

Interrogazione del db: La connessione al database avviene attraverso il file core.php che viene incluso in ogni file dove avviene la richiesta al server.

Validazione dei dati di input e sicurezza: La validazione dei dati in input viene gestita sia a lato client che a lato server, per esempio attraverso il lato client viene controllato attraverso la funzione javascript apposita sia nella pagina index.html che in index_registrazione.html che i dati vengano inseriti tutti.

Mentre nel lato server per login.php avviene un controllo per verificare le credenziali di un utente che effettua l'accesso.

Eseguendo una dichiarazione SELECT sulla tabella Iscritto per recuperare la password associata all'indirizzo email inserito la password verrà quindi memorizzata in una variabile chiamata \$hash e attraverso la funzione **password_verify()** verifichiamo se la password inserita corrisponde all'hash della password memorizzato e se la password corrisponde va avanti con le seguenti istruzioni per avviare la sessione.

Per descrivere altri aspetti dedicati alla sicurezza esiste la funzione **mysqli_real_escape_string()** in pagine come login.php e registrazione.php dove esegue l'escape di caratteri speciali in una stringa per l'utilizzo in una query MySQL, questo codice contribuisce a proteggere contro gli attacchi di SQL injection assicurando che qualsiasi carattere speciale inserito dall'utente venga trattato come un carattere letterale e non come parte di un comando SQL.

E per garantire una maggiore sicurezza attraverso il file session.php incluso in ogni file, a cui l'utente accede dopo l'inserimento delle credenziali, non è possibile accedervi in nessun modo grazie al controllo attraverso la variabile di sessione \$_SESSION["logged"], che solo attraverso l'accesso dell'utente viene impostata a true.

Presentazione: Il sito viene presentato in una chiave leggibile e intuitiva in modo da poter dare una navigazione piacevole nel sito.

Front-end:

- **Separazione presentazione/contenuto/comportamento:** si è cercato di dividere i file sulla base della modularizzazione.

Il codice dedicato alla parte CSS, javascript, ajax o sulla gestione della apertura e chiusura di sessione (sulle pagine collegate alla pagina principale e alle sue funzioni) è stato messo in un file separato all'interno di cartelle apposite sulla base del codice specifico e richiamarle in seguito nelle pagine in cui servivano attraverso il comando php "include".

- **Soluzione class-platform:** Il sito è funzionante su diverse piattaforme di browsing in quanto è stato testato su Microsoft Edge (Versione 108.0.1462.54), Google Chrome (Versione 108.0.5359.125) e Firefox (Versione 108.0.2)
- **Organizzazione file e cartelle di progetto:** I file sono stati suddivisi in cartelle per quelli basati su un linguaggio specifico da includere nel codice php.

Back-end:

- **Architettura generale classi/funzioni php:** Le funzioni php principali sono:

1. **Login.php:** Questa pagina php include un altro file PHP chiamato "core.php" all'inizio, che contiene le informazioni di base per l'accesso al database.

Quindi, il codice ottiene l'email e la password dai campi del modulo di login inviati tramite il metodo POST, e li protegge dalle injection di SQL utilizzando la funzione

"mysqli_real_escape_string" e successivamente il codice esegue una query sul database per selezionare la password dell'utente con l'email specificata.

Poi, usa la funzione "password_verify" per verificare che la password inserita dall'utente corrisponda al hash della password memorizzato nel database.

Se le credenziali sono corrette, il codice inizializza le variabili di sessione e reindirizza l'utente alla pagina principale. Altrimenti, l'utente viene reindirizzato alla pagina di login.

Infine, il codice chiude la connessione al database.

2. **Registrazione.php:** Questo file che si occupa della registrazione dell'utente ottiene il nome, il cognome, lo username, l'email e la password dai campi del modulo di registrazione inviati tramite il metodo POST e li protegge dalle injection di SQL utilizzando la funzione "mysqli_real_escape_string" (la password viene anche criptata utilizzando la funzione "password_hash").

E dopo che è stata eseguita la query viene verificato se è andata a buon fine con l'inserimento dei dati nel db, nel caso di successo avviene lo start e la creazione di variabili di sessione e l'indirizzamento verso la pagina principale, in caso contrario viene visualizzato un messaggio di errore.

3. **Ricerca.php:** Questo è un file php, incluso nella pagina principale di Mainpageutente, che si occupa di effettuare una ricerca di annunci in base ai criteri specificati dall'utente.

Nella fase iniziale di controllo verifica se è stato inserito un valore nella barra di ricerca e se c'è ottiene il valore della barra di ricerca, il valore della categoria e il valore della zona dal modulo di ricerca inviato tramite il metodo POST.

Il valore della barra di ricerca viene anche pulito utilizzando la funzione "preg_replace" per rimuovere qualsiasi carattere che non sia un numero o una lettera.

Successivamente, il codice esegue una query sul database per selezionare gli annunci che soddisfano i criteri di ricerca specificati dall'utente e la query viene costruita dinamicamente in base ai valori specificati dall'utente.

Se la query viene eseguita con successo il codice recupera i risultati e li presenta all'utente e in caso contrario presenta un messaggio di errore.

4. **Core.php:** Pagina php, inclusa nelle funzioni php che fanno uso dei dati del db, che si occupa di stabilire una connessione con il db.

- **Schema del database:** Il database contiene 5 tabelle:

1. **Iscritto**(id_iscritto,username,password,email,nome,cognome)
2. **Zona**(id_zona,nome_zona)
3. **Categoria**(id_categoria,nome)
4. **Annuncio**(id_annuncio,nome,descrizione,quantita,foto,prezzo_dollari,id_categoria,id_zona,id_categoria)

5. **Listaprefe(id_iscritto,id_annuncio)**

- **Descrizione funzioni remote:**

1. **Sessioneverifica.php:** è una funzione che gestisce la sessione di un utente.

- a) **Modo in cui usare la funzione:** la funzione deve essere inclusa nella pagina web che si vuole proteggere con l'autenticazione. Ad esempio, nel file `mainpageutente.php` viene inclusa la funzione con questo codice:
`<?php include("session/sessioneverifica.php"); ?>`.
- b) **Sintassi dell'output:** non restituisce nessun output.
- c) **Condizioni di errore:** se la sessione è scaduta o se l'utente non è autenticato, viene reindirizzato alla pagina di login (`index.php`).
- d) **Funzioni di callback lato javascript/ajax:** questa funzione non prevede l'utilizzo di callback lato javascript/ajax.

2. **Inserimentoprefe(id_annuncio):** è una funzione JavaScript che viene eseguita quando viene richiamata, accetta un parametro "id_annuncio", che viene utilizzato per inviare una richiesta AJAX al file "inserimento_lista_prefe.php".

- a) **Modo in cui usare la funzione:** La funzione "inserimentolistaprefe(id_annuncio)" può essere utilizzata chiamando la funzione con un parametro "id_annuncio" valido.

Ad esempio: "inserimentolistaprefe(1)".

Questo invierà una richiesta AJAX al file "inserimento_lista_prefe.php" con l'URL "inserimento_lista_prefe.php?id_annuncio=1".
- b) **Sintassi dell'output:** Non viene specificata alcuna sintassi dell'output nella funzione "inserimentolistaprefe(id_annuncio)".

L'output del file "inserimento_lista_prefe.php" potrebbe essere in qualsiasi formato, come ad esempio un file JSON o un file HTML.
- c) **Condizioni di errore:** Le condizioni di errore vengono gestite dalla funzione di callback nella funzione "inserimentolistaprefe(id_annuncio)".

Ad esempio, se il server non riesce a rispondere alla richiesta AJAX, viene visualizzato un messaggio di errore nella console.
- d) **Funzioni di callback lato javascript/ajax:** La funzione di callback viene eseguita quando il server risponde alla richiesta AJAX inviata dalla funzione "inserimentolistaprefe(id_annuncio)".

La funzione di callback gestisce la risposta del server, ad esempio stampandola sulla console o ricaricando la pagina corrente.